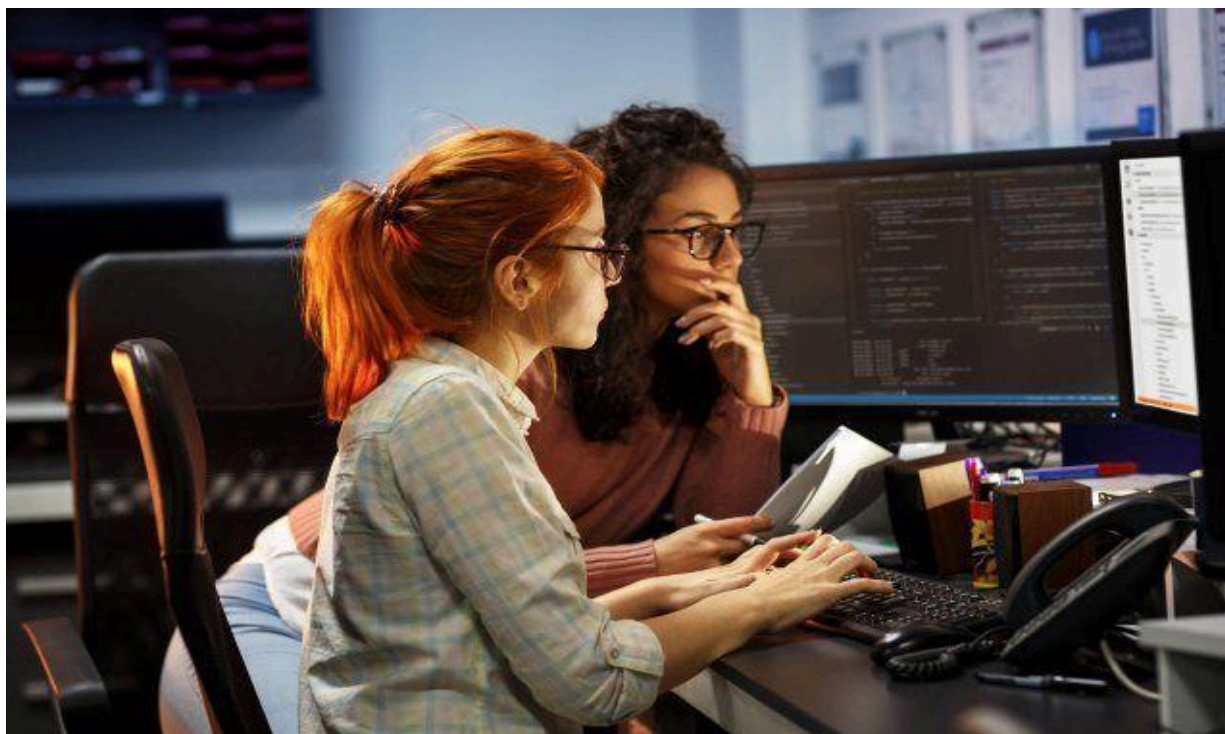




Universidad de El Salvador

Facultad Multidisciplinaria de Occidente – Ingeniería en Desarrollo de Software

Asignatura:
Programación Orientada a Objetos

Unidad #: 4

Tema: Persistencia, Archivos y Concurrencia Moderna

SEMANA 13

Autor:

Introducción:

Bienvenidos al material de lectura #2 de la Unidad #4 de la materia Programación Orientada a Objetos.

En esta unidad, exploraremos los principios fundamentales de la **Persistencia de Datos**, enfocándonos en la forma más elemental: los **Archivos Planos**. Aprenderás cómo las aplicaciones Java pueden interactuar directamente con el sistema de archivos para guardar y recuperar información, permitiendo que los datos sobrevivan más allá de la ejecución del programa.

Este documento te proporcionará una base técnica sólida, cubriendo desde la gestión de archivos y directorios con la clase `File`, hasta el manejo de flujos de texto (`FileReader`, `PrintWriter`) y flujos binarios para datos primitivos. Además, abordaremos el uso de librerías modernas como **Jackson** y **Gson** para la persistencia estructurada en formatos estándar de la industria como JSON y XML.

Al dominar estos temas, adquirirás las habilidades esenciales para el desarrollo de cualquier aplicación que necesite almacenar o intercambiar datos de manera simple y eficiente. Te animamos a experimentar activamente con los ejemplos de código para consolidar tu comprensión de estas técnicas vitales en la programación orientada a objetos.

Semana 13

4.2. Modelos básicos de persistencia (Archivos planos).

La Persistencia es la capacidad que tienen los datos generados por un programa de sobrevivir después de que este finalice su ejecución. Los Archivos Planos representan la forma de persistencia más elemental, donde los datos se almacenan de manera secuencial en un archivo de texto o binario.

En Java, la manipulación de archivos tradicionalmente se realiza a través del paquete `java.io` (Input/Output), basado en el concepto de Flujos (Streams).

4.2.1. Información sobre archivos y directorios. La clase File

Conceptos Fundamentales

En Java, la manipulación de archivos y directorios se realiza principalmente a través de dos enfoques: el **API tradicional** (`java.io.File`) y el **API moderno** (`java.nio.file.Path` y `Files`).

La clase File (java.io)

La clase `File` ha sido la herramienta tradicional para representar rutas de archivos y directorios. Aunque sigue siendo funcional, presenta algunas limitaciones en cuanto a manejo de errores y funcionalidad.

Características principales de File:

- Representa una ruta abstracta (no necesariamente un archivo existente)
- Proporciona métodos para consultar propiedades del sistema de archivos
- Permite operaciones básicas de creación y eliminación
- **Limitación importante:** Muchos métodos retornan `boolean` sin información detallada sobre errores

Constructores comunes:

```
java
```

```
File archivo = new File("ruta/archivo.txt");
File archivo2 = new File("ruta", "archivo.txt");
File archivo3 = new File(directorioFile, "archivo.txt");
```

Métodos principales de la clase File

Método	Descripción	Retorno
<code>exists()</code>	Verifica si el archivo o directorio existe	boolean
<code>isFile()</code>	Verifica si es un archivo regular	boolean
<code>isDirectory()</code>	Verifica si es un directorio	boolean
<code>getName()</code>	Obtiene el nombre del archivo	String
<code>getAbsolutePath()</code>	Obtiene la ruta absoluta	String
<code>length()</code>	Obtiene el tamaño en bytes	long
<code>createNewFile()</code>	Crea un nuevo archivo vacío	boolean
<code>mkdir()</code>	Crea un directorio	boolean
<code>mkdirs()</code>	Crea directorio incluyendo padres	boolean
<code>delete()</code>	Elimina el archivo o directorio	boolean
<code>listFiles()</code>	Lista archivos de un directorio	File[]
<code>canRead()</code>	Verifica si se puede leer	boolean
<code>canWrite()</code>	Verifica si se puede escribir	boolean

Ejemplo Básico: Operaciones con File

```
java

import java.io.File;
import java.io.IOException;

public class EjemploFile {
    public static void main(String[] args) {
        // Crear referencia a un archivo
        File archivo = new File("datos/estudiantes.txt");

        try {
            // Crear directorios padres si no existen
            File directorio = archivo.getParentFile();
            if (directorio != null && !directorio.exists()) {
                if (directorio.mkdirs()) {
                    System.out.println("Directorio creado: " + directorio.getPath());
                }
            }

            // Crear archivo si no existe
            if (archivo.createNewFile()) {
                System.out.println("Archivo creado: " + archivo.getAbsolutePath());
            } else {
                System.out.println("El archivo ya existe");
            }

            // Consultar propiedades
            System.out.println("\n=== INFORMACIÓN DEL ARCHIVO ===");
            System.out.println("Nombre: " + archivo.getName());
            System.out.println("Ruta absoluta: " + archivo.getAbsolutePath());
            System.out.println("Tamaño: " + archivo.length() + " bytes");
            System.out.println("¿Es archivo?: " + archivo.isFile());
            System.out.println("¿Es directorio?: " + archivo.isDirectory());
            System.out.println("¿Se puede leer?: " + archivo.canRead());
            System.out.println("¿Se puede escribir?: " + archivo.canWrite());

        } catch (IOException e) {
            System.err.println("Error al crear archivo: " + e.getMessage());
        }
    }
}
```

Ejemplo: Listar contenido de un directorio

```
java

import java.io.File;

public class ListarDirectorio {
    public static void main(String[] args) {
        File directorio = new File("."); // Directorio actual

        if (directorio.isDirectory()) {
            System.out.println("Contenido de: " + directorio.getAbsolutePath());
            System.out.println("-".repeat(50));

            File[] archivos = directorio.listFiles();

            if (archivos != null) {
                for (File item : archivos) {
                    String tipo = item.isDirectory() ? "[DIR]" : "[ARCHIVO]";
                    long tamaño = item.isFile() ? item.length() : 0;
                    System.out.printf("%s %-30s %10d bytes%n",
                                      tipo, item.getName(), tamaño);
                }
            }
        } else {
            System.out.println("No es un directorio válido");
        }
    }
}
```

El API moderno: Path y Files (java.nio.file)

Introducido en Java 7 y mejorado continuamente, `java.nio.file` ofrece un enfoque más robusto y expresivo para el año 2025.

Ventajas del API NIO.2:

- Mejor manejo de excepciones con información detallada
- Soporte para operaciones atómicas
- Mayor rendimiento en operaciones masivas
- Métodos más intuitivos y modernos
- Compatibilidad con diferentes sistemas de archivos

Creación de objetos Path

```
java
// Formas de crear Path (Java 11+)
Path ruta1 = Path.of("datos", "archivo.txt");
Path ruta2 = Path.of("datos/archivo.txt");

// Forma antigua (Java 7-10)
Path ruta3 = Paths.get("datos", "archivo.txt");
```

Tabla Comparativa: File vs Path/Files

Operación	API Tradicional (File)	API Moderno (Path/Files)
Crear instancia	<code>new File("ruta")</code>	<code>Path.of("ruta")</code>
Verificar existencia	<code>archivo.exists()</code>	<code>Files.exists(ruta)</code>
Crear archivo	<code>archivo.createNewFile()</code>	<code>Files.createFile(ruta)</code>
Crear directorio	<code>archivo.mkdir()</code>	<code>Files.createDirectory(ruta)</code>
Eliminar	<code>archivo.delete()</code>	<code>Files.delete(ruta)</code>
Copiar	No disponible directamente	<code>Files.copy(origen, destino)</code>
Mover	<code>archivo.renameTo()</code>	<code>Files.move(origen, destino)</code>
Tamaño	<code>archivo.length()</code>	<code>Files.size(ruta)</code>
Es directorio	<code>archivo.isDirectory()</code>	<code>Files.isDirectory(ruta)</code>
Listar contenido	<code>archivo.listFiles()</code>	<code>Files.list(ruta)</code>

Ejemplo: Operaciones con Path y Files



```

java

import java.nio.file.*;
import java.io.IOException;

public class EjemploPath {
    public static void main(String[] args) {
        // Crear referencia usando Path
        Path ruta = Path.of("datos", "estudiantes.txt");

        try {
            // Crear directorios padres
            Path directorio = ruta.getParent();
            if (directorio != null) {
                Files.createDirectories(directorio);
                System.out.println("Directorio asegurado");
            }

            // Crear archivo si no existe
            if (!Files.exists(ruta)) {
                Files.createFile(ruta);
                System.out.println("Archivo creado: " + ruta.toAbsolutePath());
            } else {
                System.out.println("El archivo ya existe");
            }

            // Consultar atributos
            System.out.println("\n=== INFORMACIÓN DEL ARCHIVO ===");
            System.out.println("Nombre: " + ruta.getFileName());
            System.out.println("Ruta absoluta: " + ruta.toAbsolutePath());
            System.out.println("Tamaño: " + Files.size(ruta) + " bytes");
            System.out.println("¿Es archivo regular?: " + Files.isRegularFile(ruta));
            System.out.println("¿Es directorio?: " + Files.isDirectory(ruta));
            System.out.println("¿Es legible?: " + Files.isReadable(ruta));
            System.out.println("¿Es escribible?: " + Files.isWritable(ruta));

        } catch (IOException e) {
            System.err.println("Error: " + e.getMessage());
            e.printStackTrace();
        }
    }
}

```

Ejemplo Avanzado: Copiar y Mover Archivos


```
import java.nio.file.*;
import java.io.IOException;

public class OperacionesAvanzadas {
    public static void main(String[] args) {
        try {
            Path origen = Path.of("datos/estudiantes.txt");
            Path destino = Path.of("respaldo/estudiantes_copia.txt");
            Path nuevo = Path.of("respaldo/estudiantes_movido.txt");

            // Asegurar que existe el directorio destino
            Files.createDirectories(destino.getParent());

            // Copiar archivo (sobreescribe si existe)
            Files.copy(origen, destino, StandardCopyOption.REPLACE_EXISTING);
            System.out.println("Archivo copiado exitosamente");

            // Mover archivo
            Files.move(destino, nuevo, StandardCopyOption.REPLACE_EXISTING);
            System.out.println("Archivo movido exitosamente");

            // Eliminar archivo
            Files.deleteIfExists(nuevo);
            System.out.println("Archivo eliminado");

        } catch (IOException e) {
            System.err.println("Error: " + e.getMessage());
        }
    }
}
```

Recomendaciones de Uso en 2025

Práctica recomendada: Utilizar `Path` y `Files` para todos los proyectos nuevos. Esta es la forma estándar y moderna de trabajar con archivos en Java.

Cuándo usar `File`: Únicamente para mantener compatibilidad con código legado o cuando se trabaja con librerías antiguas que requieren objetos `File`.

Conversión entre APIs: Si es necesario, se puede convertir entre ambos:

```
java
File archivo = new File("ruta");
Path ruta = archivo.toPath(); // File → Path
File nuevoArchivo = ruta.toFile(); // Path → File
```

Ventajas clave del API moderno para estudiantes:

1. Excepciones más descriptivas que facilitan la depuración
2. Operaciones atómicas que previenen corrupción de datos
3. Mejor integración con características modernas de Java
4. Documentación y ejemplos más actualizados en línea

4.2.2. Lectura y escritura de archivos de texto

Conceptos Fundamentales

La lectura y escritura de archivos de texto en Java implica convertir caracteres entre su representación en memoria y su forma almacenada en disco. Es fundamental considerar la **codificación de caracteres** para evitar problemas con caracteres especiales.

Codificación de Caracteres

En 2025, **UTF-8** es el estándar universal para archivos de texto. Siempre debe especificarse explícitamente la codificación al trabajar con archivos.

```
java
import java.nio.charset.StandardCharsets;

// Uso correcto: especificar UTF-8
StandardCharsets.UTF_8
```

Importante: UTF-8 garantiza compatibilidad con caracteres del español (á, é, í, ó, ú, ñ, ¿, ¡) y cualquier otro idioma.

Clases Principales para Archivos de Texto

Para Escritura

BufferedWriter: Escribe texto en un archivo utilizando un buffer intermedio que mejora el rendimiento. Es la clase recomendada para escribir archivos de texto.

PrintWriter: Proporciona métodos convenientes como `println()`, `printf()` y `format()` para escritura formateada.

Para Lectura

BufferedReader: Lee texto de manera eficiente usando un buffer. Proporciona el método `readLine()` para leer línea por línea.

Try-with-Resources: Gestión Automática de Recursos

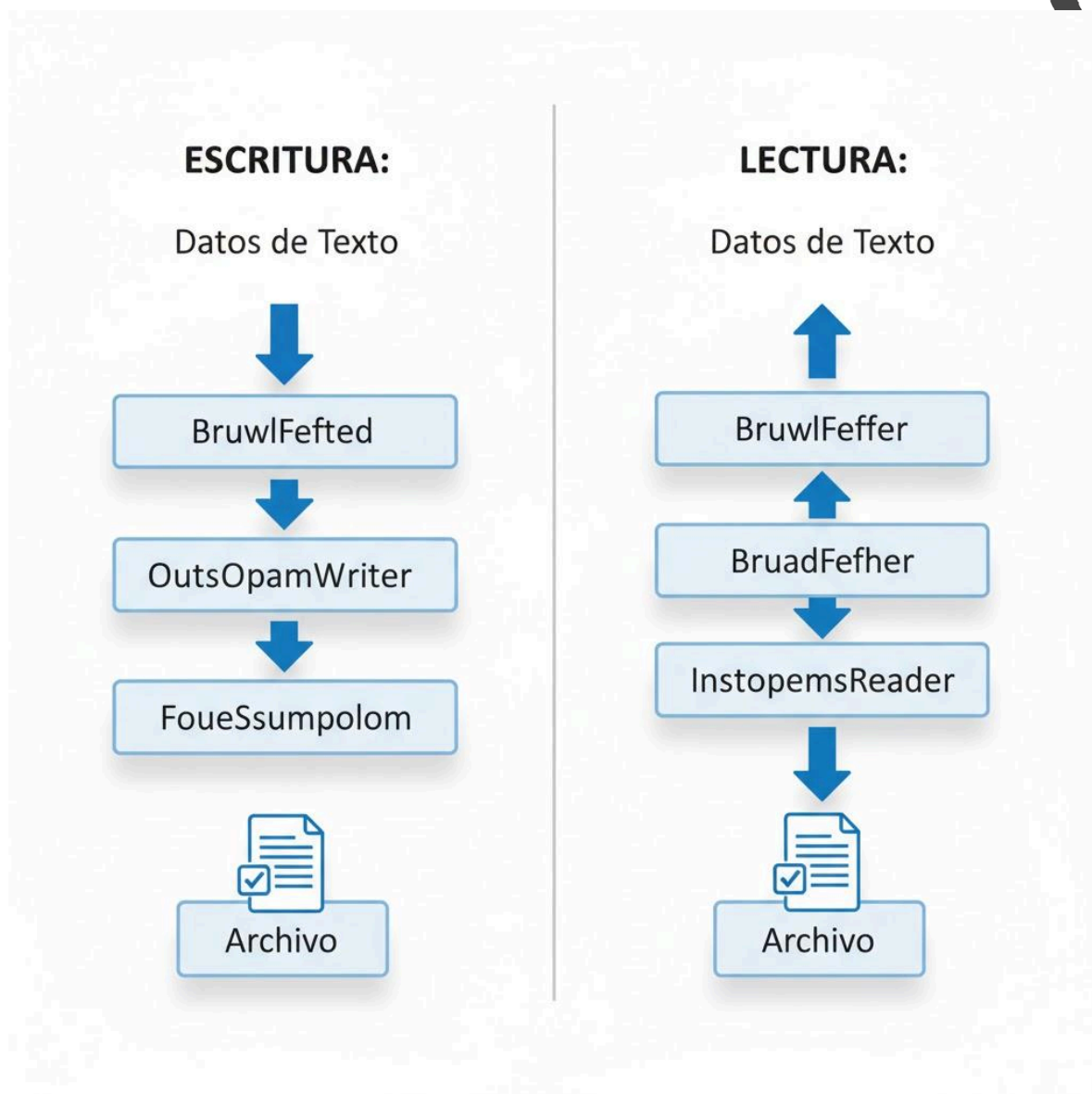
Desde Java 7, el **try-with-resources** es la forma estándar de trabajar con archivos. Garantiza el cierre automático de recursos, incluso si ocurren excepciones.

Sintaxis:

```
try (Recurso recurso = new Recurso()) {  
    // Usar el recurso  
} // Se cierra automáticamente
```

Regla de oro: Siempre use try-with-resources para operaciones de I/O. Es obligatorio en código profesional moderno.

Arquitectura de Flujos de Texto



[Diagrama de Flujo: Representación visual del flujo de datos desde la aplicación hasta el archivo en disco, mostrando cada capa de procesamiento en la cadena de escritura y lectura]

```
import java.io.*;
import java.nio.charset.StandardCharsets;

public class EscrituraBufferedWriter {
    public static void main(String[] args) {
        String nombreArchivo = "estudiantes.txt";

        try (BufferedWriter escritor = new BufferedWriter(
            new OutputStreamWriter(
                new FileOutputStream(nombreArchivo),
                StandardCharsets.UTF_8))) {

            // Escribir líneas de texto
            escritor.write("Carlos Méndez,20,Ingeniería en Sistemas");
            escritor.newLine(); // Salto de línea

            escritor.write("Ana Rodríguez,22,Desarrollo de Software");
            escritor.newLine();

            escritor.write("José Martínez,21,Tecnología");
            escritor.newLine();

            System.out.println("Datos escritos correctamente");

        } catch (IOException e) {
            System.err.println("Error al escribir: " + e.getMessage());
        }
    }
}
```

Escritura de Archivos de Texto

Método 1: Usando BufferedWriter

```

java

import java.io.*;
import java.nio.charset.StandardCharsets;

public class EscrituraBufferedWriter {
    public static void main(String[] args) {
        String nombreArchivo = "estudiantes.txt";

        try (BufferedWriter escritor = new BufferedWriter(
            new OutputStreamWriter(
                new FileOutputStream(nombreArchivo),
                StandardCharsets.UTF_8))) {

            // Escribir líneas de texto
            escritor.write("Carlos Méndez,20,Ingeniería en Sistemas");
            escritor.newLine(); // Salto de línea

            escritor.write("Ana Rodríguez,22,Desarrollo de Software");
            escritor.newLine();

            escritor.write("José Martínez,21,Tecnología");
            escritor.newLine();

            System.out.println("Datos escritos correctamente");

        } catch (IOException e) {
            System.err.println("Error al escribir: " + e.getMessage());
        }
    }
}

```

Método 2: Usando PrintWriter (Recomendado para texto formateado)

```
import java.io.*;
import java.nio.charset.StandardCharsets;

public class EscrituraPrintWriter {
    public static void main(String[] args) {
        String nombreArchivo = "calificaciones.txt";

        try (PrintWriter escritor = new PrintWriter(
            new OutputStreamWriter(
                new FileOutputStream(nombreArchivo),
                StandardCharsets.UTF_8))) {

            // println agrega automáticamente salto de línea
            escritor.println("=== REGISTRO DE CALIFICACIONES ===");
            escritor.println();

            // printf permite formateo avanzado
            escritor.printf("%-20s %10s %10s%n", "ESTUDIANTE", "PARCIAL", "FINAL");
            escritor.println("-".repeat(42));
            escritor.printf("%-20s %10.2f %10.2f%n", "María López", 8.5, 9.0);
            escritor.printf("%-20s %10.2f %10.2f%n", "Pedro Gómez", 7.8, 8.3);
            escritor.printf("%-20s %10.2f %10.2f%n", "Laura Díaz", 9.2, 9.5);

            System.out.println("Calificaciones guardadas");

        } catch (IOException e) {
            System.err.println("Error: " + e.getMessage());
        }
    }
}
```

Modo Append: Agregar al Final del Archivo

Para agregar contenido sin sobrescribir el archivo existente:

```
java

import java.io.*;
import java.nio.charset.StandardCharsets;

public class ModoAgregar {
    public static void main(String[] args) {
        String archivo = "log_sistema.txt";

        // El parámetro 'true' activa el modo append
        try (PrintWriter escritor = new PrintWriter(
            new OutputStreamWriter(
                new FileOutputStream(archivo, true), // true = append
                StandardCharsets.UTF_8))) {

            escritor.println("[INFO] Sistema iniciado - " +
                java.time.LocalDateTime.now());
            escritor.println("[INFO] Usuario conectado");

            System.out.println("Registro agregado al log");

        } catch (IOException e) {
            System.err.println("Error: " + e.getMessage());
        }
    }
}
```

Lectura de Archivos de Texto

Lectura Línea por Línea con BufferedReader


```

java

import java.io.*;
import java.nio.charset.StandardCharsets;

public class LecturaBufferedReader {
    public static void main(String[] args) {
        String nombreArchivo = "estudiantes.txt";

        try (BufferedReader lector = new BufferedReader(
            new InputStreamReader(
                new FileInputStream(nombreArchivo),
                StandardCharsets.UTF_8))) {

            String linea;
            int numeroLinea = 1;

            System.out.println("=== CONTENIDO DEL ARCHIVO ===\n");

            // Leer línea por línea hasta el final (null)
            while ((linea = lector.readLine()) != null) {
                System.out.printf("%d: %s\n", numeroLinea, linea);
                numeroLinea++;
            }

        } catch (FileNotFoundException e) {
            System.err.println("Archivo no encontrado: " + nombreArchivo);
        } catch (IOException e) {
            System.err.println("Error al leer: " + e.getMessage());
        }
    }
}

```

Clase	Método	Descripción
BufferedWriter	<code>write(String s)</code>	Escribe una cadena
BufferedWriter	<code>newLine()</code>	Escribe un salto de línea
PrintWriter	<code>println(String s)</code>	Escribe línea con salto automático
PrintWriter	<code>printf(String format, ...)</code>	Escribe con formato
BufferedReader	<code>readLine()</code>	Lee una línea (retorna null al final)
BufferedReader	<code>read()</code>	Lee un solo carácter

API Simplificada de Files (Java 11+)

Para casos simples, Java ofrece métodos directos en la clase **Files**:

```
import java.nio.file.*;
import java.nio.charset.StandardCharsets;
import java.io.IOException;
import java.util.List;

> public class FilesSimplificado {
>     public static void main(String[] args) {
        Path archivo = Path.of( first: "mensaje.txt");

        try {
            // Escribir todo el contenido de una vez
            String contenido = "Hola Mundo\nDesde Java\nEn El Salvador";
            Files.writeString(archivo, contenido, StandardCharsets.UTF_8);
            System.out.println("Archivo escrito");

            // Leer todo el contenido como String
            String leido = Files.readString(archivo, StandardCharsets.UTF_8);
            System.out.println("\nContenido leido:");
            System.out.println(leido);

            // Leer como lista de líneas
            List<String> lineas = Files.readAllLines(archivo, StandardCharsets.UTF_8);
            System.out.println("\nNúmero de líneas: " + lineas.size());

            // Escribir lista de líneas
            List<String> nuevasLineas = List.of(
                "Primera línea",
                "Segunda línea",
                "Tercera línea"
            );

            Files.write(archivo, nuevasLineas, StandardCharsets.UTF_8);
            System.out.println("Líneas escritas");

        } catch (IOException e) {
            System.err.println("Error: " + e.getMessage());
        }

    }
}
```

Cuándo usar `Files.writeString()` / `readString()`: Para archivos pequeños (menos de 1 MB) donde se necesita leer o escribir todo el contenido de una vez. No recomendado para archivos grandes por uso de memoria.

Manejo de Excepciones

```
import java.io.*;
import java.nio.charset.StandardCharsets;

public class ManejoExcepciones {
    public static void leerArchivo(String nombreArchivo) {
        try (BufferedReader lector = new BufferedReader(
            new InputStreamReader(
                new FileInputStream(nombreArchivo),
                StandardCharsets.UTF_8))) {

            String linea;
            while ((linea = lector.readLine()) != null) {
                System.out.println(linea);
            }

        } catch (FileNotFoundException e) {
            System.err.println("✗ El archivo no existe: " + nombreArchivo);
            System.err.println("    Verifique la ruta y el nombre del archivo");
        } catch (IOException e) {
            System.err.println("✗ Error de lectura: " + e.getMessage());
            e.printStackTrace();
        }
    }
}
```

Buenas Prácticas

Siempre especifique UTF-8: Use `StandardCharsets.UTF_8` en todas las operaciones de texto.

Use try-with-resources: Garantiza que los archivos se cierren correctamente.

Valide los datos: Al leer archivos, siempre valide que las líneas tengan el formato esperado antes de procesarlas.

Maneje archivos grandes con streams: Para archivos muy grandes, lea línea por línea en lugar de cargar todo en memoria.

Comente el formato: Si crea archivos con formato específico (CSV, etc.), incluya comentarios en el encabezado explicando la estructura.

Resumen de Conceptos Clave

- **BufferedReader/BufferedWriter** son las clases estándar para I/O de texto eficiente
- **PrintWriter** facilita la escritura formateada con `println()` y `printf()`
- **UTF-8** es la codificación estándar que debe usarse siempre
- **try-with-resources** cierra automáticamente los recursos y es obligatorio
- `readLine()` retorna `null` cuando alcanza el final del archivo
- Modo **append** (FileOutputStream con `true`) agrega sin sobrescribir

4.2.3. Lectura y escritura de datos primitivos en archivos

Conceptos Fundamentales

Los **archivos binarios** almacenan datos en su representación binaria nativa, sin conversión a formato de texto. A diferencia de los archivos de texto, los archivos binarios:

- **Ocupan menos espacio:** No hay conversión a caracteres ni caracteres de formato
- **Son más rápidos:** Lectura y escritura directa sin conversiones
- **Preservan exactitud:** Mantienen la precisión completa de tipos numéricos
- **No son legibles:** No pueden abrirse con editores de texto

¿Cuándo usar archivos binarios?

- Almacenar datos numéricos con precisión exacta
- Guardar grandes volúmenes de datos primitivos
- Cuando el rendimiento es crítico
- Para formatos de archivo propietarios o específicos

Clases para Datos Primitivos

DataOutputStream

Permite escribir tipos primitivos de Java directamente en formato binario. Cada tipo se escribe en su tamaño exacto en bytes.

Métodos principales:

Método	Descripción	Bytes escritos
<code>writeByte(int v)</code>	Escribe un byte	1
<code>writeShort(int v)</code>	Escribe un short	2
<code>writeInt(int v)</code>	Escribe un int	4
<code>writeLong(long v)</code>	Escribe un long	8
<code>writeFloat(float v)</code>	Escribe un float	4
<code>writeDouble(double v)</code>	Escribe un double	8
<code>writeBoolean(boolean v)</code>	Escribe un boolean	1
<code>writeChar(int v)</code>	Escribe un char	2
<code>writeUTF(String s)</code>	Escribe String en UTF-8 modificado	variable

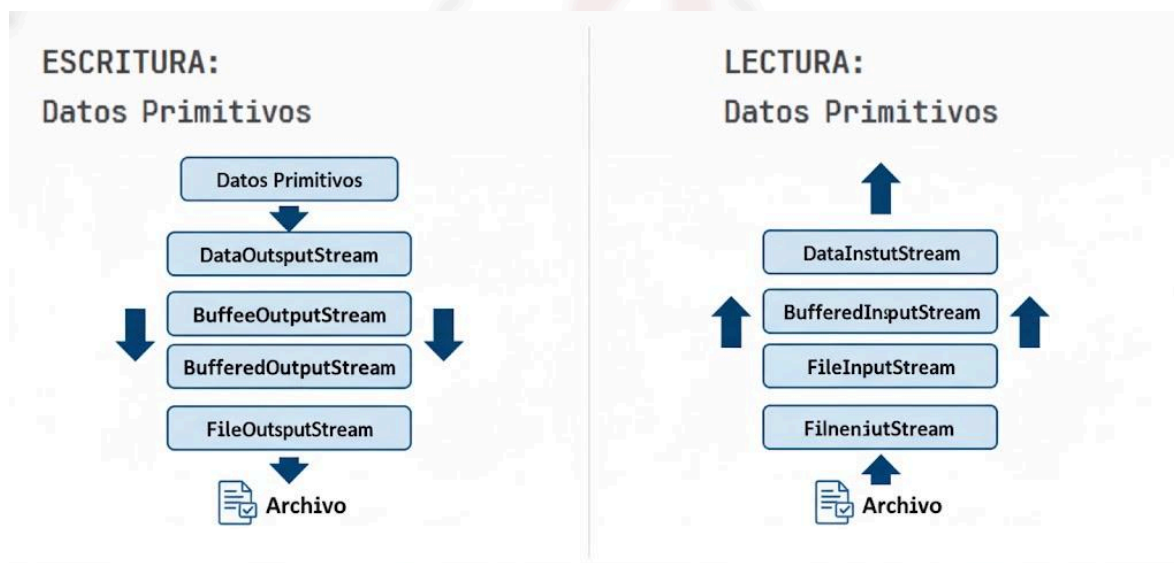
DataInputStream

Lee tipos primitivos escritos previamente por `DataOutputStream`. Es crítico leer los datos en el mismo orden y tipo en que fueron escritos.

Métodos principales:

Método	Descripción	Bytes leídos
<code>readByte()</code>	Lee un byte	1
<code>readShort()</code>	Lee un short	2
<code>readInt()</code>	Lee un int	4
<code>readLong()</code>	Lee un long	8
<code>readFloat()</code>	Lee un float	4
<code>readDouble()</code>	Lee un double	8
<code>readBoolean()</code>	Lee un boolean	1
<code>readChar()</code>	Lee un char	2
<code>readUTF()</code>	Lee un String	variable

Arquitectura de Flujos Binarios



[Diagrama de Flujo: Cadena de procesamiento de datos binarios mostrando el flujo desde tipos primitivos en memoria hasta su almacenamiento en disco, y viceversa para la lectura]

Ejemplo Básico: Escritura de Datos Primitivos

```
import java.io.*;

public class EscrituraBinaria {
    public static void main(String[] args) {
        String nombreArchivo = "estudiante.dat";

        try (DataOutputStream salida = new DataOutputStream(
            new BufferedOutputStream(
                new FileOutputStream(nombreArchivo)))) {

            // Escribir diferentes tipos de datos primitivos
            salida.writeUTF("Roberto Flores");    // String
            salida.writeInt(21);                  // Edad (int)
            salida.writeDouble(8.75);             // Promedio (double)
            salida.writeBoolean(true);            // Activo (boolean)
            salida.writeChar('M');                // Género (char)
            salida.writeLong(20230001L);          // Código (long)

            System.out.println("✓ Datos binarios escritos correctamente");

            // Mostrar tamaño del archivo
            File archivo = new File(nombreArchivo);
            System.out.println("Tamaño del archivo: " +
                archivo.length() + " bytes");

        } catch (IOException e) {
            System.err.println("Error al escribir: " + e.getMessage());
        }
    }
}
```

de Software

Ejemplo Básico: Lectura de Datos Primitivos

```
import java.io.*;

public class LecturaBinaria {
    public static void main(String[] args) {
        String nombreArchivo = "estudiante.dat";

        try (DataInputStream entrada = new DataInputStream(
            new BufferedInputStream(
                new FileInputStream(nombreArchivo)))) {

            // Leer en el MISMO ORDEN que se escribió
            String nombre = entrada.readUTF();
            int edad = entrada.readInt();
            double promedio = entrada.readDouble();
            boolean activo = entrada.readBoolean();
            char genero = entrada.readChar();
            long codigo = entrada.readLong();

            // Mostrar datos leídos
            System.out.println("=== DATOS DEL ESTUDIANTE ===");
            System.out.println("Código: " + codigo);
            System.out.println("Nombre: " + nombre);
            System.out.println("Edad: " + edad + " años");
            System.out.printf("Promedio: %.2f%n", promedio);
            System.out.println("Estado: " + (activo ? "Activo" : "Inactivo"));
            System.out.println("Género: " + genero);

        } catch (EOFException e) {
            System.err.println("Fin del archivo alcanzado inesperadamente");
        } catch (IOException e) {
            System.err.println("Error al leer: " + e.getMessage());
        }
    }
}
```



```
import java.io.*;

public class LecturaBinaria {
    public static void main(String[] args) {
        String nombreArchivo = "estudiante.dat";

        try (DataInputStream entrada = new DataInputStream(
            new BufferedInputStream(
                new FileInputStream(nombreArchivo)))) {

            // Leer en el MISMO ORDEN que se escribió
            String nombre = entrada.readUTF();
            int edad = entrada.readInt();
            double promedio = entrada.readDouble();
            boolean activo = entrada.readBoolean();
            char genero = entrada.readChar();
            long codigo = entrada.readLong();

            // Mostrar datos leídos
            System.out.println("=== DATOS DEL ESTUDIANTE ===");
            System.out.println("Código: " + codigo);
            System.out.println("Nombre: " + nombre);
            System.out.println("Edad: " + edad + " años");
            System.out.printf("Promedio: %.2f\n", promedio);
            System.out.println("Estado: " + (activo ? "Activo" : "Inactivo"));
            System.out.println("Género: " + genero);

        } catch (EOFException e) {
            System.err.println("Fin del archivo alcanzado inesperadamente");
        } catch (IOException e) {
            System.err.println("Error al leer: " + e.getMessage());
        }
    }
}
```

Crítico: Los datos deben leerse en el **mismo orden exacto** en que fueron escritos. Si se intenta leer un `int` donde se escribió un `double`, los datos se corromperán.

Tabla: Tamaños de Tipos Primitivos

Tipo Java	Tamaño (bytes)	Rango de valores
byte	1	-128 a 127
short	2	-32,768 a 32,767
int	4	-2,147,483,648 a 2,147,483,647
long	8	-9,223,372,036,854,775,808 a 9,223,372,036,854,775,807
float	4	$\pm 3.4 \times 10^{38}$ (precisión ~7 dígitos)
double	8	$\pm 1.7 \times 10^{308}$ (precisión ~15 dígitos)
boolean	1	true o false
char	2	0 a 65,535 (caracteres Unicode)

Manejo de EOFException

Cuando se lee un archivo binario sin saber cuántos registros contiene, se usa **EOFException** para detectar el final:



```
import java.io.*;

public class LecturaConEOF {
    public static void main(String[] args) {
        String archivo = "numeros.dat";

        try (DataInputStream entrada = new DataInputStream(
            new BufferedInputStream(
                new FileInputStream(archivo)))) {

            int contador = 0;
            double suma = 0;

            try {
                // Leer hasta que lance EOFException
                while (true) {
                    double numero = entrada.readDouble();
                    System.out.println("Número " + (contador + 1) + ": " + numero);
                    suma += numero;
                    contador++;
                }
            } catch (EOFException e) {
                // Fin del archivo - comportamiento esperado
                System.out.println("\n✓ Fin del archivo alcanzado");
                System.out.println("Total de números: " + contador);
                if (contador > 0) {
                    System.out.printf("Promedio: %.2f%n", suma / contador);
                }
            }

            } catch (IOException e) {
                System.err.println("Error: " + e.getMessage());
            }
        }
    }
}
```

Ejemplo: Almacenamiento de Datos Científicos

```
import java.io.*;

public class DatosCientificos {

    public static void guardarMediciones(String archivo, 1 usage
                                         double[] temperaturas, long[] timestamps) {

        try (DataOutputStream salida = new DataOutputStream(
            new BufferedOutputStream(
                new FileOutputStream(archivo)))) {

            // Guardar cantidad de mediciones
            salida.writeInt(temperaturas.length);

            // Guardar cada par temperatura-timestamp
            for (int i = 0; i < temperaturas.length; i++) {
                salida.writeDouble(temperaturas[i]);
                salida.writeLong(timestamps[i]);
            }

            System.out.println("✓ " + temperaturas.length + " mediciones guardadas");

        } catch (IOException e) {
            System.err.println("Error: " + e.getMessage());
        }
    }

    public static void analizarMediciones(String archivo) { 1 usage
        try (DataInputStream entrada = new DataInputStream(
            new BufferedInputStream(
                new FileInputStream(archivo)))) {
```

```
int cantidad = entrada.readInt();

double suma = 0;
double minima = Double.MAX_VALUE;
double maxima = Double.MIN_VALUE;

System.out.println("\n=== ANÁLISIS DE MEDICIONES ===");
System.out.println("Total de registros: " + cantidad);
System.out.println("\nDetalle:");

for (int i = 0; i < cantidad; i++) {
    double temp = entrada.readDouble();
    long timestamp = entrada.readLong();

    suma += temp;
    if (temp < minima) minima = temp;
    if (temp > maxima) maxima = temp;

    System.out.printf("Registro %d: %.2f°C (timestamp: %d)%n",
        i + 1, temp, timestamp);
}

System.out.println("\n--- ESTADÍSTICAS ---");
System.out.printf("Temperatura promedio: %.2f°C%n", suma / cantidad);
System.out.printf("Temperatura mínima: %.2f°C%n", minima);
System.out.printf("Temperatura máxima: %.2f°C%n", maxima);
} catch (IOException e) {
    System.err.println("Error: " + e.getMessage());
}
```

```
        System.err.println("Error: " + e.getMessage());
    }
}

public static void main(String[] args) {
    String archivoMediciones = "temperaturas.dat";

    // Datos de ejemplo
    double[] temperaturas = {23.5, 24.1, 22.8, 25.3, 23.9, 24.7};
    long[] timestamps = {
        System.currentTimeMillis(),
        System.currentTimeMillis() + 3600000,
        System.currentTimeMillis() + 7200000,
        System.currentTimeMillis() + 10800000,
        System.currentTimeMillis() + 14400000,
        System.currentTimeMillis() + 18000000
    };

    // Guardar mediciones
    guardarMediciones(archivoMediciones, temperaturas, timestamps);

    // Analizar mediciones
    analizarMediciones(archivoMediciones);
}
```

Ingeniería en Desarrollo
de Software

Buenas Prácticas

Orden consistente: Documente el orden y tipo de datos en comentarios. Crear una especificación del formato del archivo.

Versiónado: Incluya un número de versión al inicio del archivo para manejar cambios futuros en el formato.

Cantidad de registros: Guarde el número de registros al inicio del archivo para facilitar la lectura.

Buffering: Siempre use `BufferedInputStream` y `BufferedOutputStream` para mejorar significativamente el rendimiento.

EOFException: Use `EOFException` solo cuando no sepa cuántos datos hay. Si conoce la cantidad, use un contador.

Resumen de Conceptos Clave

- **DataOutputStream/DataInputStream** escriben y leen tipos primitivos en formato binario
- Los datos deben leerse en el **mismo orden y tipo** exacto en que se escribieron
- Los archivos binarios son más eficientes en espacio y velocidad que los de texto
- `EOFException` indica que se alcanzó el final del archivo
- Siempre use buffering para mejorar el rendimiento
- Documente el formato del archivo binario claramente

4.2.4. Manejo de archivos JSON/XML con Jackson y Gson

Conceptos Fundamentales

Los formatos **JSON** (JavaScript Object Notation) y **XML** (eXtensible Markup Language) son estándares para el intercambio de datos estructurados. A diferencia de los archivos de texto plano o binarios, estos formatos permiten representar estructuras de datos complejas de manera legible y estandarizada.

Características de JSON:

- Formato ligero y fácil de leer
- Ampliamente utilizado en APIs web y servicios REST
- Sintaxis simple basada en pares clave-valor
- Tipos de datos: objetos, arrays, strings, números, booleanos, null

Características de XML:

- Formato más verboso pero muy estructurado
- Soporte robusto para validación con esquemas
- Ampliamente usado en sistemas empresariales y legacy
- Permite namespaces y atributos complejos

Librerías Java para JSON y XML en 2025

Para JSON:

Gson (Google):

- Librería madura y estable de Google
- API simple e intuitiva
- Excelente para casos de uso básicos y medianos
- Buen rendimiento general

Jackson:

- Librería más completa y poderosa
- Mayor rendimiento en operaciones masivas
- Más opciones de configuración y personalización
- Estándar de facto en Spring Framework
- **Recomendada para proyectos profesionales en 2025**

Para XML:

Jackson XML:

- Extensión de Jackson para XML
- API consistente con Jackson JSON
- Buena integración con el ecosistema Jackson

JAXB (Java Architecture for XML Binding):

- Parte del JDK hasta Java 10, ahora librería externa
- Muy usado en sistemas empresariales

Recomendación 2025: Use **Jackson** tanto para JSON como para XML, ya que proporciona una API consistente y es el estándar industrial actual.

Configuración de Dependencias

Para usar estas librerías, debe agregarlas a su proyecto. En un proyecto con Maven:


```

<!-- pom.xml -->
<dependencies>
  <!-- Jackson para JSON -->
  <dependency>
    <groupId>com.fasterxml.jackson.core</groupId>
    <artifactId>jackson-databind</artifactId>
    <version>2.17.0</version>
  </dependency>

  <!-- Jackson para XML -->
  <dependency>
    <groupId>com.fasterxml.jackson.dataformat</groupId>
    <artifactId>jackson-dataformat-xml</artifactId>
    <version>2.17.0</version>
  </dependency>

  <!-- Gson (alternativa para JSON) -->
  <dependency>
    <groupId>com.google.code.gson</groupId>
    <artifactId>gson</artifactId>
    <version>2.10.1</version>
  </dependency>
</dependencies>

```

Para proyectos sin Maven, descargue los archivos JAR y agréguelos al classpath.

[Diagrama de Flujo: Proceso de serialización (Objeto Java → JSON/XML → Archivo) y deserialización (Archivo → JSON/XML → Objeto Java)]

Trabajando con JSON usando Jackson

Estructura básica de JSON

```

json
{
  "nombre": "Carlos Méndez",
  "edad": 21,
  "activo": true,
  "calificaciones": [8.5, 9.0, 7.8],
  "direccion": {
    "ciudad": "San Salvador",
    "pais": "El Salvador"
  }
}

```

Ejemplo Básico: Clase de Datos (POJO)

```
public class Estudiante { no usages
    private String nombre; 4 usages
    private int edad; 4 usages
    private String carrera; 4 usages
    private double promedio; 4 usages
    private boolean activo; 4 usages

    // Constructor vacío (requerido para Jackson)
    public Estudiante() { no usages
    }

    // Constructor con parámetros
    public Estudiante(String nombre, int edad, String carrera, no usages
                        double promedio, boolean activo) {
        this.nombre = nombre;
        this.edad = edad;
        this.carrera = carrera;
        this.promedio = promedio;
        this.activo = activo;
    }

    // Getters y Setters (requeridos para Jackson)
    public String getNombre() { return nombre; } no usages
    public void setNombre(String nombre) { this.nombre = nombre; } no u

    public int getEdad() { return edad; } no usages
    public void setEdad(int edad) { this.edad = edad; } no usages

    public String getCarrera() { return carrera; } no usages
    public void setCarrera(String carrera) { this.carrera = carrera; }
```

```
public double getPromedio() { return promedio; } no usages
public void setPromedio(double promedio) { this.promedio = promedio; } no usages

public boolean isActive() { return activo; } no usages
public void setActive(boolean activo) { this.activo = activo; } no usages

@Override
public String toString() {
    return String.format("%s (%d años) - %s - Promedio: %.2f - %s",
        nombre, edad, carrera, promedio, activo ? "Activo" : "Inactivo");
}
```

Escritura de JSON con Jackson



```

import com.fasterxml.jackson.databind.ObjectMapper;
import com.fasterxml.jackson.databind.SerializationFeature;
import java.io.File;
import java.io.IOException;

public class EscrituraJSON {
    public static void main(String[] args) {
        // Crear el ObjectMapper (motor de Jackson)
        ObjectMapper mapper = new ObjectMapper();

        // Configurar formato legible (identado)
        mapper.enable(SerializationFeature.INDENT_OUTPUT);

        // Crear objeto estudiante
        Estudiante estudiante = new Estudiante(
            "María López",
            20,
            "Ingeniería en Sistemas",
            8.75,
            true
        );

        try {
            // Escribir objeto a archivo JSON
            File archivo = new File("estudiante.json");
            mapper.writeValue(archivo, estudiante);
            System.out.println("✓ Archivo JSON creado: " + archivo.getAbsolutePath());

            // También se puede obtener como String
            String json = mapper.writeValueAsString(estudiante);
            System.out.println("\nContenido JSON:");
            System.out.println(json);

        } catch (IOException e) {
            System.err.println("Error al escribir JSON: " + e.getMessage());
        }
    }
}

```

Lectura de JSON con Jackson

```
import com.fasterxml.jackson.databind.ObjectMapper;
import java.io.File;
import java.io.IOException;

public class LecturaJSON {
    public static void main(String[] args) {
        ObjectMapper mapper = new ObjectMapper();

        try {
            // Leer objeto desde archivo JSON
            File archivo = new File("estudiante.json");
            Estudiante estudiante = mapper.readValue(archivo, Estudiante.class);

            System.out.println("✓ Archivo JSON leído correctamente");
            System.out.println("\nDatos del estudiante:");
            System.out.println(estudiante);

            // También se puede leer desde String
            String json = "{\"nombre\":\"Pedro Gómez\",\"edad\":22,\" +
                \"carrera\":\"Desarrollo\",\"promedio\":9.1,\" +
                \"activo\":true}";
            Estudiante estudiante2 = mapper.readValue(json, Estudiante.class);
            System.out.println("\nDesde String JSON:");
            System.out.println(estudiante2);

        } catch (IOException e) {
            System.err.println("Error al leer JSON: " + e.getMessage());
        }
    }
}
```

de Software

Trabajando con Listas en JSON

```
import com.fasterxml.jackson.databind.ObjectMapper;
import com.fasterxml.jackson.databind.SerializationFeature;
import com.fasterxml.jackson.core.type.TypeReference;
import java.io.File;
import java.io.IOException;
import java.util.ArrayList;
import java.util.List;

public class JSONconListas {

    // Guardar lista de estudiantes
    public static void guardarEstudiantes(String archivo, 1 usage
                                         List<Estudiante> estudiantes) {

        ObjectMapper mapper = new ObjectMapper();
        mapper.enable(SerializationFeature.INDENT_OUTPUT);

        try {
            mapper.writeValue(new File(archivo), estudiantes);
            System.out.println("✓ " + estudiantes.size() +
                               " estudiantes guardados en " + archivo);
        } catch (IOException e) {
            System.err.println("Error: " + e.getMessage());
        }
    }

    // Cargar lista de estudiantes
    public static List<Estudiante> cargarEstudiantes(String archivo) { 1 usage
        ObjectMapper mapper = new ObjectMapper();
        List<Estudiante> estudiantes = new ArrayList<>();

        try {
            // TypeReference especifica el tipo genérico
            estudiantes = mapper.readValue(new File(archivo),
                                           new TypeReference<List<Estudiante>>() {});

            System.out.println("✓ " + estudiantes.size() +
                               " estudiantes cargados desde " + archivo);
        } catch (IOException e) {
            System.err.println("Error: " + e.getMessage());
        }

        return estudiantes;
    }
}
```

```
public static void main(String[] args) {  
    String archivoJSON = "estudiantes.json";  
  
    // Crear lista de estudiantes  
    List<Estudiante> estudiantes = new ArrayList<>();  
    estudiantes.add(new Estudiante("Ana Flores", 21,  
        "Ingeniería", 8.9, true));  
    estudiantes.add(new Estudiante("Carlos Ramos", 20,  
        "Sistemas", 9.2, true));  
    estudiantes.add(new Estudiante("Laura Díaz", 22,  
        "Desarrollo", 8.5, false));  
  
    // Guardar  
    guardarEstudiantes(archivoJSON, estudiantes);  
  
    // Cargar  
    List<Estudiante> cargados = cargarEstudiantes(archivoJSON);  
  
    // Mostrar  
    System.out.println("\n=== LISTA DE ESTUDIANTES ===");  
    cargados.forEach(System.out::println);  
}
```

Anotaciones Jackson para Personalización

```
import com.fasterxml.jackson.annotation.*;
import java.time.LocalDate;

public class EstudianteAvanzado { no usages

    @JsonProperty("nombre_completo") // Cambiar nombre en JSON 3 usages
    private String nombre;

    private int edad; 3 usages

    @JsonIgnore // No incluir en JSON 2 usages
    private String password;

    @JsonFormat(pattern = "dd/MM/yyyy") // Formato de fecha 2 usages
    private LocalDate fechaIngreso;

    @JsonInclude(JsonInclude.Include.NON_NULL) // Solo si no es null 2 usages
    private String telefono;

    // Constructor, getters, setters...

    public EstudianteAvanzado() {} no usages

    public EstudianteAvanzado(String nombre, int edad) { no usages
        this.nombre = nombre;
        this.edad = edad;
    }

    // Getters y Setters
    public String getNombre() { return nombre; } no usages
    public void setNombre(String nombre) { this.nombre = nombre; } no usages

    public int getEdad() { return edad; } no usages
    public void setEdad(int edad) { this.edad = edad; } no usages

    public String getPassword() { return password; } no usages
    public void setPassword(String password) { this.password = password; } no usages

    public LocalDate getFechaIngreso() { return fechaIngreso; } no usages
    public void setFechaIngreso(LocalDate fechaIngreso) { no usages
        this.fechaIngreso = fechaIngreso;
    }
}
```



```
public String getTelefono() { return telefono; } no usages  
public void setTelefono(String telefono) { this.telefono = telefono; }
```

Trabajando con JSON usando Gson

Gson es una alternativa más simple a Jackson, ideal para proyectos pequeños.



```
import com.google.gson.Gson;
import com.google.gson.GsonBuilder;
import com.google.gson.reflect.TypeToken;
import java.io.*;
import java.lang.reflect.Type;
import java.util.ArrayList;
import java.util.List;

public class EjemploGson {

    // Guardar objeto con Gson
    public static void guardarConGson(String archivo, Estudiante estudiante) { 1 usage
        // Crear Gson con formato legible
        Gson gson = new GsonBuilder().setPrettyPrinting().create();

        try (Writer writer = new FileWriter(archivo)) {
            gson.toJson(estudiante, writer);
            System.out.println("✓ Guardado con Gson: " + archivo);
        } catch (IOException e) {
            System.err.println("Error: " + e.getMessage());
        }
    }

    // Cargar objeto con Gson
    public static Estudiante cargarConGson(String archivo) { 1 usage
        Gson gson = new Gson();

        try (Reader reader = new FileReader(archivo)) {
            Estudiante estudiante = gson.fromJson(reader, Estudiante.class);
            System.out.println("✓ Cargado con Gson");
            return estudiante;
        } catch (IOException e) {
            System.err.println("Error: " + e.getMessage());
            return null;
        }
    }

    // Guardar lista con Gson
    public static void guardarListaGson(String archivo, no usages
        List<Estudiante> estudiantes) {
        Gson gson = new GsonBuilder().setPrettyPrinting().create();

        try (Writer writer = new FileWriter(archivo)) {
            gson.toJson(estudiantes, writer);
        }
    }
}
```

```
        System.out.println("✓ Lista guardada con Gson");
    } catch (IOException e) {
        System.err.println("Error: " + e.getMessage());
    }
}

// Cargar lista con Gson
public static List<Estudiante> cargarListaGson(String archivo) {    no usages
    Gson gson = new Gson();

    try (Reader reader = new FileReader(archivo)) {
        Type tipoLista = new TypeToken<ArrayList<Estudiante>>(){}.getType();
        List<Estudiante> estudiantes = gson.fromJson(reader, tipoLista);
        System.out.println("✓ Lista cargada con Gson");
        return estudiantes;
    } catch (IOException e) {
        System.err.println("Error: " + e.getMessage());
        return new ArrayList<>();
    }
}

public static void main(String[] args) {
    Estudiante est = new Estudiante("Roberto Silva", 23,
        "TI", 8.8, true);

    // Guardar y cargar
    guardarConGson( archivo: "estudiante_gson.json", est);
    Estudiante cargado = cargarConGson( archivo: "estudiante_gson.json");
    System.out.println(cargado);
}
```

Tabla Comparativa: Jackson vs Gson

Característica	Jackson	Gson
Rendimiento	Más rápido	Bueno
API	Más compleja	Más simple
Personalización	Muy flexible	Limitada
Streaming	Sí	Limitado
Tamaño librería	Mayor	Menor
Adopción industrial	Muy alta	Alta
Mejor para	Proyectos profesionales	Proyectos pequeños/medianos

Buenas Prácticas

Jackson como estándar: Para proyectos profesionales en 2025, use Jackson por su rendimiento y flexibilidad.

Gson para simplicidad: Use Gson en proyectos educativos o pequeños donde la simplicidad es prioritaria.

Constructores vacíos: Siempre incluya un constructor sin parámetros para que Jackson/Gson puedan crear instancias.

Getters y Setters: Son requeridos para que las librerías accedan a los campos privados.

Formato legible: Use `INDENT_OUTPUT` en desarrollo para facilitar la depuración.

Validación: Siempre valide los datos después de deserializar para garantizar integridad.

Manejo de errores: Capture `IOException` y proporcione mensajes de error claros.

Resumen de Conceptos Clave

- **JSON** es el formato preferido para intercambio de datos en 2025
- **Jackson** es la librería estándar industrial para JSON y XML
- **Gson** es una alternativa más simple para proyectos pequeños
- La **serialización** convierte objetos Java a JSON/XML
- La **deserialización** convierte JSON/XML a objetos Java
- Use `ObjectMapper` (Jackson) o `Gson` como instancias reutilizables
- Los POJOs requieren constructor vacío, getters y setters
- `TypeReference` se usa para deserializar colecciones genéricas

